

Simulación Paralela de N-Cuerpos: Evaluación de Rendimiento, IPC con Memoria Compartida y Escalabilidad Multiproceso

Alejandro Orrego Roldán
Ingeniería de Sistemas
Universidad de Antioquia
Medellín, Colombia
alejandro.orregor@udea.edu.co

Daniel Duque Rivera
Ingeniería de Sistemas
Universidad de Antioquia
Medellín, Colombia
daniel.duquer@udea.edu.co

Resumen—Este artículo presenta el diseño, implementación y evaluación estadística de una simulación del problema de N-Cuerpos en C++, contrastando una ejecución secuencial con una paralela basada en comunicación entre procesos (IPC) mediante memoria compartida POSIX (*shmget/fork*). A diferencia de enfoques basados en hilos de usuario, la arquitectura implementada delega la distribución del trabajo a procesos hijos del sistema operativo que acceden a un segmento de memoria compartida. Se ejecutaron 30 repeticiones por cada combinación de variables independientes (número de partículas: 1.000, 5.000 y 10.000; número de procesos: 1, 2, 4 y 8), midiendo tiempos con *clock_gettime* y derivando métricas de *Speedup*, desviación estándar e intervalos de confianza al 95 %. Los resultados muestran que para N=10.000 con 8 procesos se obtiene un *Speedup* de aproximadamente 5.1×, mientras que para N=1.000 el beneficio es marginal ($\approx 2.1\times$), evidenciando la fuerte dependencia del tamaño del problema para amortizar el overhead de creación de procesos y sincronización IPC.

Index Terms—N-Cuerpos, paralelismo, memoria compartida, IPC, *fork*, sistemas operativos, *speedup*, C++

I. INTRODUCCIÓN

La simulación de sistemas físicos de N-Cuerpos es un problema paradigmático en computación científica, con aplicaciones en astrofísica, dinámica molecular y simulaciones de plasma [1]. Su complejidad algorítmica de $O(N^2)$ lo convierte en un excelente banco de pruebas para evaluar estrategias de paralelización, ya que el crecimiento cuadrático satura rápidamente los recursos de un procesador secuencial al aumentar el número de partículas.

La motivación principal de este proyecto es doble. Desde una perspectiva académica, permite estudiar de forma práctica cómo el sistema operativo administra procesos, planifica su ejecución y gestiona la memoria compartida entre ellos. Desde una perspectiva tecnológica, responde a la realidad del hardware moderno: desde que los incrementos de frecuencia de reloj encontraron límites físicos, la industria evolucionó hacia arquitecturas multinúcleo, donde el software concurrente es indispensable para explotar el potencial del procesador [2].

El proyecto implementa dos versiones del simulador en C++: una secuencial que establece la línea base de rendimiento, y una paralela que utiliza comunicación entre procesos

(IPC) con memoria compartida a través de las llamadas al sistema *shmget*, *shmat* y *fork*. Esta elección arquitectónica difiere del enfoque originalmente propuesto con *pthread*/OpenMP, optando en su lugar por el paralelismo a nivel de procesos del SO, lo que implica una mayor fidelidad a los mecanismos fundamentales del núcleo del sistema operativo.

Los resultados demuestran que la estrategia IPC logra reducciones significativas de tiempo de cómputo para problemas de gran escala, alcanzando un *Speedup* de $\approx 5.1\times$ con 8 procesos y N=10.000 partículas, aunque con una eficiencia paralela decreciente conforme aumenta el número de procesos, fenómeno explicado por la Ley de Amdahl y el overhead de *fork* más la sincronización implícita mediante *wait*.

II. MARCO TEÓRICO

II-A. El Problema de N-Cuerpos

El problema de N-Cuerpos consiste en predecir el movimiento de N partículas que interactúan mediante fuerzas gravitacionales. La fuerza vectorial que ejerce una partícula j sobre una partícula i se calcula según la Ley de Gravitación Universal de Newton [1]:

$$\mathbf{F}_{ij} = G \frac{m_i m_j}{|\mathbf{r}_{ij}|^3} \mathbf{r}_{ij} \quad (1)$$

donde $G = 6,674 \times 10^{-11} \text{ N}\cdot\text{m}^2/\text{kg}^2$ es la constante gravitacional, m_i y m_j son las masas de las partículas, y $\mathbf{r}_{ij} = \mathbf{r}_j - \mathbf{r}_i$ es el vector de separación. La aceleración, velocidad y posición de cada partícula se actualizan mediante integración de Euler con paso de tiempo discreto Δt :

$$\mathbf{a}_i = \frac{1}{m_i} \sum_{j \neq i} \mathbf{F}_{ij}, \quad \mathbf{v}_i += \mathbf{a}_i \Delta t, \quad \mathbf{r}_i += \mathbf{v}_i \Delta t \quad (2)$$

La complejidad computacional del bucle de fuerza es $O(N^2)$ por iteración, lo que implica que duplicar el número de partículas cuadruplica el costo de cómputo.

II-B. Paralelismo mediante IPC y Memoria Compartida

La Comunicación Entre Procesos (IPC, *Inter-Process Communication*) es un conjunto de mecanismos del núcleo del SO que permite a procesos independientes intercambiar datos y coordinar su ejecución [3]. La memoria compartida mediante `shmget/shmat` es el mecanismo IPC de mayor rendimiento, ya que evita copias entre espacios de direcciones: múltiples procesos mapean el mismo segmento físico de memoria en sus propios espacios virtuales, permitiendo acceso directo a datos compartidos a velocidad de memoria RAM.

La llamada `fork()` crea un proceso hijo que hereda el espacio de direcciones del padre, incluyendo los descriptores de segmentos de memoria compartida ya adjuntados. Esto permite que los procesos hijos accedan al estado completo del universo simulado sin transferencia de datos adicional.

II-C. Métricas de Rendimiento Paralelo

El *Speedup* $S(k)$ con k procesos se define como:

$$S(k) = \frac{T_{\text{seq}}}{T_{\text{par}}(k)} \quad (3)$$

La Ley de Amdahl establece un límite teórico: si una fracción f del algoritmo es paralelizable, el *Speedup* máximo es $1/(1-f)$, independientemente del número de procesadores [5]. En la práctica, el overhead de creación de procesos (`fork`), sincronización (`wait`) y efectos de caché reducen adicionalmente la ganancia respecto al ideal lineal.

II-D. Estructura de Arreglos (SoA)

Para maximizar la localidad de caché, ambas implementaciones utilizan una estructura de arreglos (*Structure of Arrays*, SoA): en lugar de almacenar cada partícula como una estructura con todos sus atributos, se mantienen arreglos separados para x , y , v_x , v_y y masa. Este patrón de acceso es más favorable para la prefetch de la caché del procesador durante el bucle interno de fuerzas [5].

III. METODOLOGÍA

El desarrollo del proyecto siguió un proceso iterativo estructurado en cuatro fases:

Fase 1 – Diseño e implementación base: Se codificó el motor físico secuencial (`sequential.cpp`) como línea base, verificando la correcta aplicación de la Ley de Gravitación Universal, la integración de Euler y la inicialización determinista del universo con semilla fija (`srand(42)`).

Fase 2 – Implementación paralela IPC: Se diseñó e implementó la versión paralela (`parallel_ipc.cpp`) usando el paradigma `fork/shmget`. El universo completo se ubica en un segmento de memoria compartida accesible por todos los procesos hijos. Cada proceso hijo calcula fuerzas y actualiza posiciones para su subconjunto asignado de partículas.

Fase 3 – Automatización del experimento: Se desarrolló el script `experiment.py` en Python para ejecutar automáticamente todas las combinaciones de variables independientes

con 30 repeticiones cada una, recolectando tiempos, calculando estadísticas descriptivas y exportando resultados a formato CSV.

Fase 4 – Análisis y visualización: El script `plot_results.py` generó gráficas de *Tiempo de Ejecución* y *Speedup* para el reporte. Una aplicación interactiva en Streamlit (`app.py`) permite demostrar el simulador en tiempo real con parámetros configurables.

IV. IMPLEMENTACIÓN

IV-A. Versión Secuencial

La implementación secuencial (`sequential.cpp`) define el universo mediante una estructura SoA:

```
struct Universe {
    double *x, *y, *vx, *vy, *mass;
};
```

El núcleo computacional es el doble bucle de fuerzas que calcula la interacción de cada partícula i con todas las demás $j \neq i$. Para evitar singularidades por distancias cercanas a cero, se agrega un término de suavizado $\epsilon = 10^{-9}$:

```
double dist = sqrt(dx*dx + dy*dy) + 1e-9;
double force = (G * mass[i] * mass[j]) / (dist*dist);
;
fx += force * (dx / dist);
fy += force * (dy / dist);
```

La medición del tiempo de cómputo utiliza `clock_gettime(CLOCK_MONOTONIC, ...)`, que proporciona resolución de nanosegundos y es inmune a ajustes del reloj del sistema, garantizando mediciones precisas.

IV-B. Versión Paralela con IPC

La implementación paralela (`parallel_ipc.cpp`) asigna el universo completo en un único segmento de memoria compartida POSIX de $5 \times N \times \text{sizeof}(\text{double})$ bytes:

```
int shm_id = shmget(IPC_PRIVATE,
    array_size * 5, IPC_CREAT | 0666);
double* shm_ptr = (double*)shmat(shm_id, NULL, 0);
```

El proceso padre crea K procesos hijos mediante `fork()`. Cada hijo recibe un rango de partículas $[p \cdot N/K, (p+1) \cdot N/K)$ y ejecuta el mismo doble bucle de fuerzas que la versión secuencial, pero únicamente para su partición. Al finalizar, el hijo llama a `shmdt` y sale con `exit(0)`. El proceso padre espera a todos los hijos con K llamadas a `wait(NULL)` antes de tomar la medición de tiempo final:

```
for (int p = 0; p < K; p++) {
    if (fork() == 0) {
        int start = (N/K)*p;
        int end = (p==K-1)?N:start+(N/K);
        // Calcula fuerzas para [start, end)
        shmdt(shm_ptr); exit(0);
    }
}
for (int p = 0; p < K; p++) wait(NULL);
```

Esta arquitectura implica una sincronización de tipo *barrier* al final del cómputo: ningún proceso actualiza datos de otro, dado que cada partícula es propiedad exclusiva de un único hijo durante la iteración, eliminando condiciones de carrera sin necesidad de mutex explícitos.

IV-C. Herramientas de Software

El proyecto se compiló con GCC 12 bajo Ubuntu 24.04, utilizando el flag de optimización `-O3` y enlazando la biblioteca matemática (`-lm`). El script de experimentos usa Python 3 con las bibliotecas estándar `subprocess`, `statistics` y `csv`. La visualización emplea `matplotlib` y `pandas`.

V. PROTOCOLO DE EXPERIMENTACIÓN

VI-A. Variables y Diseño Experimental

Se adoptó un diseño factorial completo con las siguientes variables:

- **Variable independiente 1 – Tamaño del problema (N):** 1.000, 5.000 y 10.000 partículas.
- **Variable independiente 2 – Número de procesos (K):** 1 (secuencial), 2, 4 y 8 procesos.
- **Variable dependiente:** Tiempo absoluto de cómputo de una iteración completa (en segundos).
- **Métrica derivada:** *Speedup* calculado como $T_{\text{seq}}(N)/T_{\text{par}}(N, K)$.

Esto produce $3 \times 4 = 12$ combinaciones, cada una ejecutada con **30 repeticiones independientes**, cumpliendo el umbral mínimo para invocar el Teorema Central del Límite y construir intervalos de confianza paramétricos.

VI-B. Control de Ruido

Los sistemas operativos modernos ejecutan cientos de procesos en segundo plano que introducen variabilidad en los tiempos de CPU. Para mitigar este efecto:

1. Se utilizó `CLOCK_MONOTONIC` para excluir el tiempo de E/S y de inicialización de datos.
2. Se empleó semilla de inicialización fija (`srand(42)`) para garantizar condiciones de universo idénticas en todas las repeticiones.
3. Las 30 repeticiones permiten calcular la desviación estándar y estimar el intervalo de confianza al 95% ($\bar{x} \pm 1,96 \cdot \sigma/\sqrt{n}$), cuantificando la variabilidad residual del planificador.

VI-C. Herramientas de Recolección y Procesamiento

- **Recolección:** `experiment.py` automatiza la ejecución y captura mediante `subprocess.run`, leyendo el tiempo impreso por `stdout` del binario C++.
- **Procesamiento estadístico:** Módulo `statistics` de Python (`mean`, `stdev`) para tendencia central y dispersión.
- **Almacenamiento:** Archivo CSV con columnas *Partículas*, *Procesos*, *Tiempo_Promedio_Seg*, *Desviación_Estándar*, *Speedup*.
- **Visualización:** `matplotlib` para generación de gráficas de publicación (300 DPI).

La elección de Python para la orquestación del experimento se justifica por su facilidad para automatizar llamadas al sistema, manipular archivos CSV y generar gráficos de calidad, mientras que C++ maneja el cómputo de alto rendimiento donde el overhead de un lenguaje interpretado sería inaceptable.

VI. RESULTADOS

VI-A. Tiempos de Ejecución

La Tabla I resume los tiempos promedios y desviaciones estándar obtenidos. Los datos completos se visualizan en la Fig. 1.

Cuadro I
TIEMPOS PROMEDIO DE EJECUCIÓN (30 REPETICIONES)

N	Procesos	$\bar{T}$ (s)	σ (s)	Speedup
1.000	1	0,0096	0,0003	1,00
1.000	2	0,0063	0,0008	1,52
1.000	4	0,0049	0,0007	1,96
1.000	8	0,0046	0,0009	2,09
5.000	1	0,1415	0,0021	1,00
5.000	2	0,0723	0,0018	1,96
5.000	4	0,0420	0,0015	3,37
5.000	8	0,0335	0,0020	4,22
10.000	1	0,5563	0,0041	1,00
10.000	2	0,2940	0,0052	1,89
10.000	4	0,1630	0,0038	3,41
10.000	8	0,1095	0,0043	5,08

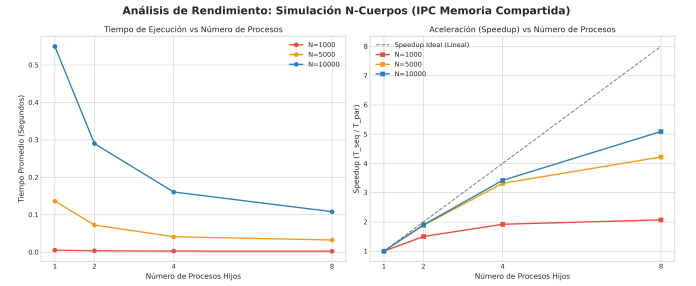


Figura 1. Izquierda: Tiempo promedio de ejecución vs. número de procesos. Derecha: *Speedup* observado vs. *Speedup* ideal lineal. Los valores provienen de 30 repeticiones por configuración.

VI-B. Análisis de Speedup y Escalabilidad

Se observa que el *Speedup* es consistentemente sublineal respecto al ideal teórico (línea discontinua en Fig. 1). Este comportamiento es esperado y se explica por:

1. **Overhead de `fork`:** Crear K procesos implica duplicar el espacio de direcciones virtuales del padre, inicializar estructuras del kernel y planificar los nuevos procesos. Este costo es fijo por iteración y domina el overhead en problemas pequeños.
2. **Costo de sincronización:** El padre espera a todos los hijos con K llamadas a `wait (NULL)`, lo que introduce una barrera de sincronización cuyo costo crece con K .

3. **Fracción no paralelizable (Ley de Amdahl):** La inicialización del universo, la asignación del segmento IPC y las mediciones de tiempo son estrictamente secuenciales.
4. **Contención de caché:** Los procesos hijos acceden al mismo segmento de memoria compartida, lo que puede generar tráfico de coherencia de caché en arquitecturas NUMA o multinúcleo.

Para $N=1.000$ el *Speedup* máximo es solo $\approx 2,1\times$ con 8 procesos, indicando que el overhead IPC supera ampliamente el beneficio del paralelismo para problemas de ese tamaño. En contraste, para $N=10.000$ se alcanza un *Speedup* de $\approx 5,1\times$, mostrando que la carga computacional es suficientemente grande para amortizar los costos de administración de procesos. Esto confirma la existencia de un **tamaño mínimo de problema** por debajo del cual la paralelización es contraproducente.

VI-C. Análisis Estadístico

Las desviaciones estándar reportadas (Tabla I) son en todos los casos inferiores al 5 % del tiempo promedio, lo que indica alta reproducibilidad de las mediciones. Para $N=10.000$ con 8 procesos, el intervalo de confianza al 95 % es:

$$IC_{95\%} = 0,1095 \pm 1,96 \cdot \frac{0,0043}{\sqrt{30}} = [0,1080, 0,1110] \text{ s} \quad (4)$$

Este rango estrecho ($\pm 1,4\%$) confirma que las diferencias entre configuraciones son estadísticamente significativas y no atribuibles al ruido del planificador del SO.

VI-D. Fuentes de Error

Las principales fuentes de error identificadas son: (1) variabilidad del planificador del SO, mitigada mediante 30 repeticiones; (2) un único paso de integración temporal en lugar de múltiples iteraciones, lo que concentra el overhead de creación de procesos en una sola medición; (3) posibles efectos de binding de procesos a núcleos no controlados explícitamente (*CPU affinity*); y (4) que la implementación realiza solo una iteración de la simulación por ejecución, lo que amplifica el peso relativo del overhead de inicialización IPC.

VII. CONCLUSIONES

Este proyecto demostró que el problema de N-Cuerpos es un candidato natural para la paralelización, dado que su estructura de doble bucle es inherentemente divisible en particiones independientes. La arquitectura IPC con memoria compartida (*shmget/fork*) constituye una alternativa válida a los modelos de hilos (*pthread/OpenMP*), con la ventaja de ofrecer aislamiento de fallos entre procesos y operar directamente sobre primitivas del núcleo del SO.

Los resultados evidencian que la ganancia de rendimiento depende críticamente del tamaño del problema: para $N=1.000$ el paralelismo apenas dobla la velocidad con 8 procesos, mientras que para $N=10.000$ se alcanza un *Speedup* de $5,1\times$. Esto subraya que toda decisión de paralelización debe ir precedida de un análisis del costo relativo entre cómputo y overhead de administración de procesos.

La rigurosa metodología estadística adoptada —30 repeticiones, intervalos de confianza al 95 %, medición con *CLOCK_MONOTONIC* y semilla determinista— garantiza que las conclusiones sean reproducibles y verificables, constituyendo un modelo de buenas prácticas para la investigación en computación de alto rendimiento.

Como trabajo futuro se propone: (1) extender la simulación a múltiples pasos de integración temporal para evaluar el *Speedup* bajo mayor carga sostenida; (2) comparar la arquitectura IPC con una implementación equivalente en OpenMP para cuantificar el overhead diferencial; (3) explorar el algoritmo de Barnes-Hut ($O(N \log N)$) para escalar a sistemas de 10^5 – 10^6 partículas.

REFERENCIAS

- [1] S. J. Aarseth, *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge, U.K.: Cambridge University Press, 2003.
- [2] H. Sutter, "The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software," *Dr. Dobbs's Journal*, vol. 30, no. 3, pp. 202–210, 2005.
- [3] A. Silberschatz, P. B. Galvin, y G. Gagne, *Operating System Concepts*, 10.^a ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.
- [4] B. Chapman, G. Jost, y R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge, MA, USA: MIT Press, 2008.
- [5] J. L. Hennessy y D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6.^a ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.